TECHNICAL REFERENCE & GUIDE

Stage 1.4.2.5.4 — Docling Table / Image / Formula Understanding

Understanding Pipeline Options, Crop Generation Behavior, and Multimodal Extraction

> **Finding Summary:** Yes — `<class 'NoneType'>` is expected with the way we currently configured Docling. This is a good finding for Stage 1.4.2.5.4.

The important point is:

- `len(doc.pictures) > 0` means Docling **detected the picture region**.
- `picture.get_image(doc)` returning `None` means Docling **did not generate/store the cropped picture image** in the DoclingDocument.

Docling's PDF pipeline has a specific option called `generate_picture_images`, which controls whether individual picture crops are generated. It defaults to `False`. (GitHub)

Step 1 — Why this happened

Our current code was simply:

```
converter = DocumentConverter()
result = converter.convert(pdf_path)
doc = result.document
```

So effectively:

```
PDF
 ↓
Docling
 ↓
Detect picture
 ↓
PictureItem created
 ↓
generate_picture_images = False
 ↓
picture.image = None
```

Therefore:

```
picture = doc.pictures[0]
picture_image = picture.get_image(doc)
print(type(picture_image))
```

produces:

```
<class 'NoneType'>
```

> **Key Takeaway:** This does not mean that Docling failed to detect your architecture diagram. It means Docling knows that a picture exists, but we didn't ask the PDF pipeline to generate the actual picture image. This distinction is extremely important.

Step 2 — Let's verify this

Run:

```
picture = doc.pictures[0]
print(picture)
```

You will likely see something conceptually like:

```
PictureItem(
    ...
    label=picture,
    prov=[...],
    image=None,
    ...
)
```

The important part is `image=None`. The picture has been detected and has provenance information such as its page and bounding box, but the actual image object isn't populated. This behavior has also been reported in Docling's own issue/discussion history. (GitHub)

Step 3 — Configure Docling to generate picture images

Now let's modify our converter. Add:

```
from docling.datamodel.pipeline_options import PdfPipelineOptions
from docling.datamodel.base_models import InputFormat
from docling.document_converter import (
    DocumentConverter,
    PdfFormatOption,
)
```

Then:

```
pipeline_options = PdfPipelineOptions()
pipeline_options.generate_picture_images = True
```

And create the converter:

```
converter = DocumentConverter(
    format_options={
        InputFormat.PDF: PdfFormatOption(
            pipeline_options=pipeline_options
        )
    }
)
```

Now convert again:

```
result = converter.convert(pdf_path)
doc = result.document
```

Step 4 — Test the picture again

```
print("Number of pictures:", len(doc.pictures))
```

Then:

```
picture = doc.pictures[0]
picture_image = picture.get_image(doc)
print(type(picture_image))
```

This time we expect:

```
<class 'PIL.Image.Image'>
```

instead of `<class 'NoneType'>`.

Step 5 — Save the extracted picture

Now:

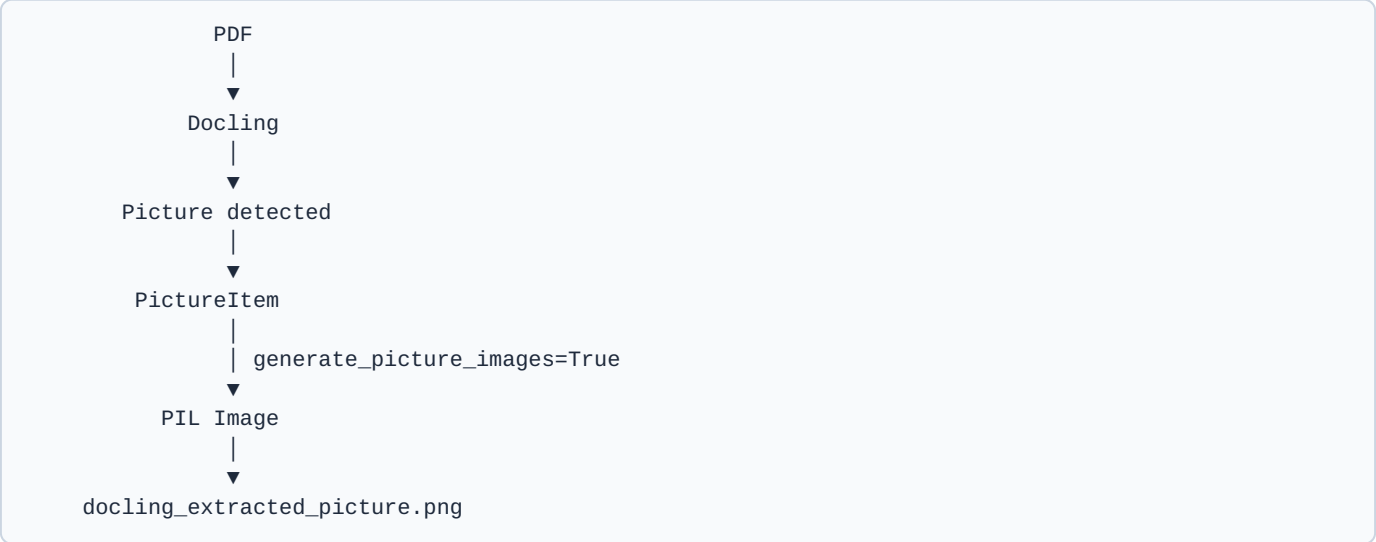
```
from pathlib import Path

output_image = Path(
    "docling_extracted_picture.png"
)

picture_image.save(output_image)

print(
    "Saved:",
    output_image
)
```

You have now demonstrated the complete flow:



Docling's own example for exporting figures uses `element.get_image(conv_res.document)` after enabling the relevant image generation pipeline configuration. (GitHub)

Step 6 — Let's make the experiment more interesting

Instead of extracting only the first picture, let's extract every picture and table image.

```
from pathlib import Path

output_dir = Path("docling_extracted_elements")
output_dir.mkdir(exist_ok=True)

# ------------------------------------------------------------
# Extract pictures
# ------------------------------------------------------------

for index, picture in enumerate(doc.pictures, start=1):
    image = picture.get_image(doc)
    print(
        f"Picture {index}:",
        type(image)
    )

    if image is not None:
        image_path = (
            output_dir /
            f"picture_{index}.png"
        )
        image.save(image_path)
        print(
            "  Saved:",
            image_path
        )
```

And tables:

```
# ------------------------------------------------------------
# Extract table images
# ------------------------------------------------------------

for index, table in enumerate(doc.tables, start=1):
    image = table.get_image(doc)
    print(
        f"Table {index}:",
        type(image)
    )

    if image is not None:
        image_path = (
            output_dir /
            f"table_{index}.png"
        )
        image.save(image_path)
        print(
            "  Saved:",
            image_path
        )
```

Now we're learning an important Docling capability:

```
DoclingDocument
│
├── tables
│    └── TableItem
│         └── get_image()
│
└── pictures
     └── PictureItem
          └── get_image()
```

Step 7 — One important distinction

This is worth writing into your notebook.

> `doc.pictures` answers: **"What pictures did Docling identify?"**
>
> `picture.get_image(doc)` answers: **"Can I obtain the actual picture image from the DoclingDocument?"**

These are different questions. Therefore, `len(doc.pictures)` can be `1` while `picture.get_image(doc)` can still return `None`. That's exactly what happened to you.

Step 8 — What about our formula?

This also gives us a useful insight. Our sample formula was intentionally inserted as an image:

```
P(A|B) = P(B|A)P(A) / P(B)
```

Therefore, with our current PDF, Docling may see it as **Picture** rather than **Formula**. That's actually useful for the experiment. We want to distinguish:

```
Image containing a formula
        ≠
Semantic formula recognition
```

Later we'll enable Docling's formula enrichment and compare the results. Docling provides a `do_formula_enrichment` pipeline option specifically for formula recognition/LaTeX enrichment. (GitHub)

Our current Stage 1.4.2.5.4 status

This is actually a successful experiment, not an error.

Stage 1.4.2.5.4 Sub-step	Description	Status
Step 1	Create complex test PDF	✅ **Complete**
Step 2	Convert using Docling	✅ **Complete**
Step 3	Detect table	✅ **Complete**
Step 4	Detect picture	✅ **Complete**
Step 5	Extract picture	🔵 **We found the configuration issue**
Step 6	Generate picture image	⏭️ **Now**
Step 7	Understand formula handling	⏭️ **Next**
Step 8	Compare all representations	⏭️ **Next**

> **The key lesson from your NoneType:**
> Docling's detection of a picture and generation of its actual image are separate operations. And that's exactly the kind of thing we want to discover through hands-on experimentation rather than simply memorizing Docling APIs.

Run Steps 3–5 above with `generate_picture_images=True`. Once you get `PIL.Image.Image`, we'll move to the formula enrichment experiment, which is the next logical part of Stage 1.4.2.5.4.